

Back Propagation Neural Network for XOR Function

Aim

To implement a Back Propagation Neural Network for solving the XOR problem using binary input and output.

Objectives

- Understand the concept of Back Propagation.
- Implement XOR using a neural network.
- Train the network using gradient descent.
- Visualize error reduction during training.

Theory

Back Propagation is a supervised learning algorithm used in Artificial Neural Networks.

XOR is a non-linear problem and cannot be solved using a single-layer perceptron. Therefore, a hidden layer is used.

The neural network performs forward propagation, error calculation, backpropagation, and weight updating.

Network Architecture

Input Layer : 2 Neurons
Hidden Layer : 2 Neurons
Output Layer : 1 Neuron

XOR Truth Table

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Important Formulas

1. Sigmoid Function:

$$\sigma(x) = 1 / (1 + e^{-x})$$

2. Sigmoid Derivative:

$$\sigma'(x) = x(1 - x)$$

3. Weight Update Rule:

$$W(\text{new}) = W(\text{old}) + \text{Learning Rate} \times \text{Error} \times \text{Input}$$

Efficient Python Code

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[0]])

sigmoid = lambda x: 1/(1+np.exp(-x))
derivative = lambda x: x*(1-x)

np.random.seed(1)
w1 = np.random.rand(2,2)
w2 = np.random.rand(2,1)

errors = []

for i in range(10000):

    h = sigmoid(np.dot(X,w1))
    o = sigmoid(np.dot(h,w2))

    e = Y-o
    errors.append(np.mean(np.abs(e)))

    d_o = e * derivative(o)
    d_h = d_o.dot(w2.T) * derivative(h)

    w2 += h.T.dot(d_o) * 0.1
    w1 += X.T.dot(d_h) * 0.1

print(o)

plt.plot(errors)
plt.title("Error Reduction")
plt.xlabel("Epochs")
plt.ylabel("Error")
plt.grid()
plt.show()
```

Code Explanation

- NumPy is used for matrix operations.
- Matplotlib is used for graph visualization.
- XOR input and output datasets are defined.
- Sigmoid activation function is used.
- Forward propagation computes output.
- Backpropagation reduces error by updating weights.

How to Run (.py File in Anaconda)

1. Open Anaconda Navigator.
2. Launch Spyder or VS Code.
3. Save code as xor_backprop.py.
4. Open Anaconda Prompt.
5. Install libraries using:
pip install numpy matplotlib
6. Run using:
python xor_backprop.py

How to Run (.ipynb File in Anaconda)

1. Open Anaconda Navigator.
2. Launch Jupyter Notebook.
3. Open the notebook file.

4. Run each cell using Shift + Enter.
5. Output and graph will appear automatically.

Expected Output

```
[0.02]  
[0.98]  
[0.98]  
[0.03]
```

Outcome

The neural network successfully learns the XOR function and reduces error during training.

Conclusion

The Back Propagation Neural Network correctly classifies XOR inputs and demonstrates supervised learning.

Important Viva Questions with Answers

Q. What is Back Propagation?

Ans. It is a supervised learning algorithm used to train neural networks.

Q. Why is XOR non-linear?

Ans. Because it cannot be separated by a straight line.

Q. Why is hidden layer needed?

Ans. To solve non-linear problems like XOR.

Q. What is an epoch?

Ans. One complete training cycle.

Q. What does sigmoid function do?

Ans. It converts values into outputs between 0 and 1.

Q. What does the graph show?

Ans. Reduction of error during training.

Q. Which libraries are used?

Ans. NumPy and Matplotlib.

Q. What is learning rate?

Ans. It controls the speed of learning.